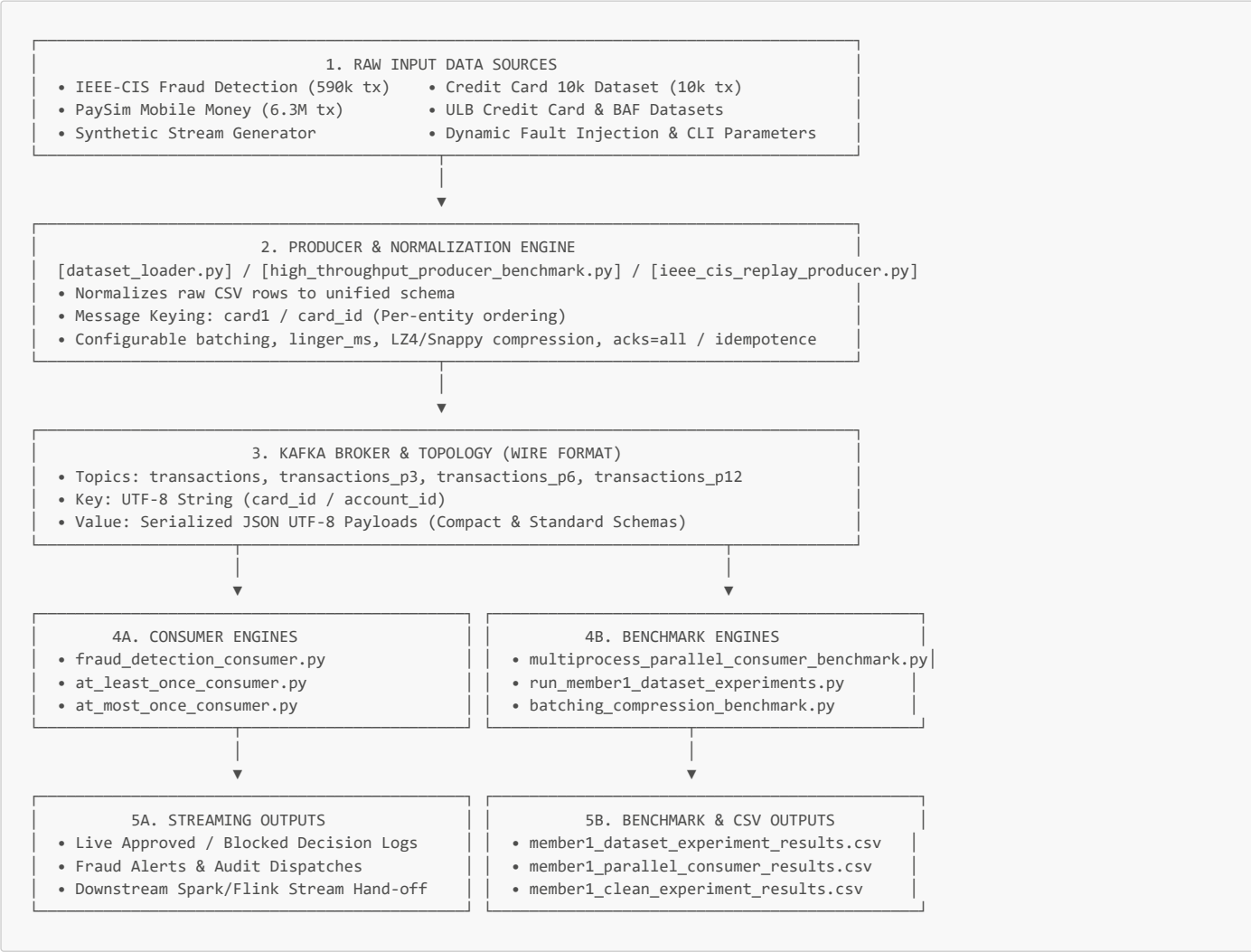


Comprehensive Input and Output Specification

This document provides an exhaustive, end-to-end specification of all **Inputs** and **Outputs** across every component, dataset, producer, Kafka topic, consumer engine, and benchmark in this codebase.

1. System Architecture & End-to-End Data Flow



2. Input Specifications (Data Ingestion Layer)

2.1 File-Based Dataset Inputs (Datasets/)

The codebase ingests real-world financial transaction datasets from the `Datasets/` directory:

Dataset Identifier	File Location	Record Count	Features	Primary Ground Truth Target
IEEE-CIS Fraud Detection	Datasets/IEEE CIS-.../IEEE CIS/train_transaction.csv or Datasets/raw/train_transaction.csv	590,540 rows	434 columns	isFraud (0 = Legitimate, 1 = Fraud)
Credit Card 10k	Datasets/Credit card Fraud detection/credit_card_fraud_10k.csv	10,000 rows	10 columns	is_fraud (0 = Legitimate, 1 = Fraud)
PaySim Mobile Money	Datasets/Paysim/paysim dataset.csv	6,362,620 rows	11 columns	isFraud (0 = Legitimate, 1 = Fraud)
ULB Credit Card	Datasets/ulb_creditcard/creditcard.csv	284,807 rows	31 columns (PCA V1-V28, Time, Amount)	Class (0 = Legitimate, 1 = Fraud)
Bank Account Fraud (BAF)	Datasets/BAF/Base.csv	1,000,000 rows	32 columns	fraud_bool (0 or 1)

Detailed Schema of Primary Datasets:

A. IEEE-CIS Fraud Detection (train_transaction.csv)

- **TransactionID** (int): Unique sequential transaction identifier (e.g., 2987000).
- **isFraud** (int): Binary target classification ground truth (0 or 1).
- **TransactionDT** (int): Timedelta from reference date (seconds).
- **TransactionAmt** (float): Transaction payment amount in USD (e.g., 68.50).
- **ProductCD** (string): Product / merchant code (e.g., 'W', 'H', 'C', 'R').
- **card1 - card6** (string/float): Card categorical payment metadata (e.g., card1: 13926, card4: discover, card6: credit).
- **addr1, addr2** (string/float): Billing region and country codes (e.g., addr2: 87).
- **P_emaildomain, R_emaildomain** (string): Purchaser and recipient email domain.
- **C1-C14, D1-D15, M1-M9, V1-V339**: Quantitative identity, risk, match, and behavioral features.

B. Credit Card 10k (credit_card_fraud_10k.csv)

- **transaction_id** (int/string): Sequential ID (1 to 10000).
- **amount** (float): Transaction monetary value (e.g., 124.50).
- **cardholder_age** (int): Age of cardholder.
- **merchant_category** (string): Category (e.g., 'Retail', 'Electronics', 'Travel').
- **device_trust_score** (int): Client device trust rating (0 to 100).
- **location_mismatch** (int): Binary flag (1 = IP location differs from billing country).
- **foreign_transaction** (int): Binary flag (1 = International charge).
- **velocity_last_24h** (int): Count of transactions on card in last 24 hours.
- **is_fraud** (int): Target label (0 or 1).

2.2 Dynamic Runtime & CLI Execution Inputs

When running producer or consumer scripts, inputs are supplied via command-line flags and runtime parameters:

```
# Example Producer Execution Input
python kafka/producer/high_throughput_producer_benchmark.py \
  --topic transactions_p6 \
  --workers 4 \
  --batch-size 128 \
  --linger-ms 15 \
  --acks all \
  --max-records 50000

# Example Consumer Execution Input
python kafka/consumer/fraud_detection_consumer.py \
  --broker localhost:9092 \
  --topic transactions \
  --group fraud-detection-engine-group \
  --threshold 5000.0 \
  --max-messages 1000
```

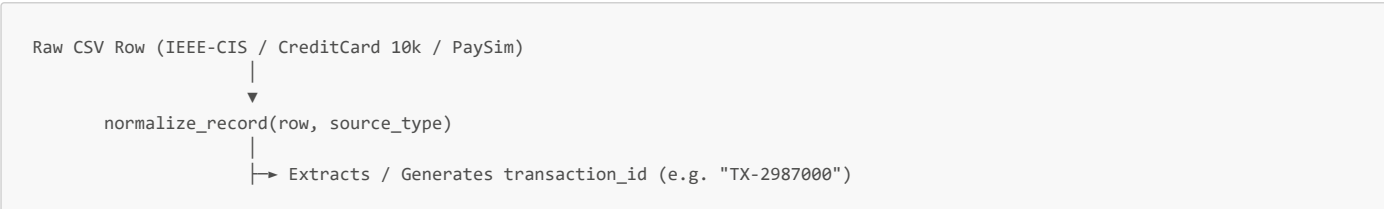
Key Producer CLI Input Flags:

- **--broker**: Kafka broker connection string (default: localhost:9092).
- **--topic**: Target Kafka topic (transactions, transactions_p3, transactions_p6, transactions_p12).
- **--dataset**: Dataset selection key (ieee_cis, credit_card_10k, paysim, ulb, baf, or custom file path).
- **--rate**: Target transaction emission rate in transactions per second (100, 500, 1000, 5000, 0 for unconstrained max speed).
- **--batch-size**: Producer micro-batch buffer size in Kilobytes (16, 64, 128, 256).
- **--linger-ms**: Milliseconds to buffer before dispatching TCP batch (0, 5, 15, 50).
- **--compression**: Message compression codec (none, lz4, snappy, gzip, zstd).
- **--acks**: Acknowledgment durability level (0, 1, all).
- **--workers**: Number of parallel OS processes for producer ingestion (1, 2, 4, 8).

3. Transformation & Ingestion Layer (Producer Stage)

The normalization engine (kafka/transaction_generator/dataset_loader.py) parses diverse dataset rows into standardized Python dictionaries before JSON serialization.

Normalization Logic Flow



- ↳ Extracts / Generates card_id (e.g. "CARD-13926")
- ↳ Converts amount to rounded float (e.g. 68.50)
- ↳ Generates ISO-8601 & Epoch timestamps
- ↳ Normalizes fraud target flag to boolean
- ↳ Maps merchant category, device type, country

4. Intermediate Kafka Message Wire Contract

This is the exact data contract transmitted through Apache Kafka brokers.

4.1 Kafka Record Key (Partition Affinity Contract)

- **Type:** `String` (UTF-8 Encoded)
- **Value:** Entity Identifier (`card1`, `card_id`, or `nameOrig`)
- **Purpose:** Guarantees that all transactions for the same card or user hash to the **exact same partition**. Ensures strict chronological ordering and enables stateful velocity aggregations without distributed shuffles.

4.2 Kafka Record Value (Payload Schemas)

Schema Option 1: Standard Real-Time Event Payload (`transactions` topic)

```
{
  "transaction_id": "TX-2987000",
  "card_id": "CARD-13926",
  "amount": 68.50,
  "merchant_id": "M-W",
  "timestamp": "2026-10-05T14:56:50.123456+00:00",
  "epoch_timestamp": 1791212210.123,
  "device_type": "mobile",
  "country": "US",
  "is_fraud": false,
  "dataset_source": "ieee_cis",
  "card2": "150.0",
  "card4": "discover",
  "card6": "credit",
  "P_emaildomain": "gmail.com"
}
```

Schema Option 2: High-Throughput Compact Payload (`ieee_cis_transactions` topic)

Optimized for high-throughput benchmarks (29,000+ tx/s) and downstream Spark/Flink ML inference:

```
{
  "TransactionID": 2987000,
  "isFraud": 0,
  "TransactionDT": 86400,
  "TransactionAmt": 68.50,
  "card1": "13926",
  "ProductCD": "W",
  "timestamp": 1791212210.123
}
```

5. Consumer & Analytics Processing Layer

5.1 Real-Time Fraud Rule Evaluation Engine (`fraud_detection_consumer.py`)

Input to Consumer:

Kafka streaming messages from topic `transactions`.

Rule Engine Evaluation Logic:

1. **Ground Truth Flag:** Checks if `is_fraud == True` or `isFraud == 1`.
2. **High-Value Threshold:** Evaluates if `amount >= threshold` (default: \$5,000.00).
3. **Geographic Risk:** Checks if `location_mismatch == True` or `country == 'FOREIGN'`.
4. **Card Velocity:** Checks if `velocity_24h > 5` rapid bursts.

Output Decision:

- **Approved:** Transaction verified under normal thresholds.

- **Suspicious / Fraudulent Alert:** Transaction blocked, flagged with specific violation reasons, and routed for audit.

6. Output Specifications (Result & Artifact Layer)

6.1 Real-Time Console & Alert Outputs

When `fraud_detection_consumer.py` processes transactions, it outputs structured real-time inspection records:

```
=====
REAL-TIME FINANCIAL FRAUD DETECTION CONSUMER (DATASET-DRIVEN)
=====
Broker           : localhost:9092
Topic            : transactions
Consumer Group   : fraud-detection-engine-group
Amount Threshold : $5000.00
Max Messages     : Continuous
=====

-----
Transaction ID : TX-2987000 | Source: ieee_cis | Part: 2 | Offset: 1042
Card / User    : CARD-13926 | Amount: $68.50 | Merchant: M-W
Device / Geo   : mobile | Country: US

[+] Transaction Verified (Normal Behavior)
Status         : APPROVED
Kafka Offset 1042 securely committed.
-----

Transaction ID : TX-3012450 | Source: credit_card_10k | Part: 1 | Offset: 1043
Card / User    : CARD-HOLDER-45-120 | Amount: $7850.00 | Merchant: CAT-Electronics
Device / Geo   : mobile | Country: FOREIGN

[!] [ALERT] SUSPICIOUS / FRAUDULENT TRANSACTION DETECTED!
* Flagged fraudulent in benchmark ground truth dataset
* High transaction amount exceeds safety threshold ($7850.00 >= $5000.00)
* Geographic/location mismatch detected
Status         : BLOCKED / SENT TO FRAUD AUDIT
Kafka Offset 1043 securely committed.

>>> [LIVE STATS] Processed: 50 | Fraud Detected: 3 (6.0%) | Total Vol: $42,150.00 <<<
```

Final Session Summary Output:

```
=====
FINAL SESSION SUMMARY
=====
Total Transactions Processed : 1000
Fraudulent / Flagged Events  : 42 (4.20%)
Total Volume Monitored      : $284,930.50
=====
```

6.2 Experiment Benchmark CSV Output Artifacts

The repository generates 4 structured CSV output files during experimental benchmark runs:

1. `member1_dataset_experiment_results.csv`

Contains throughput and latency percentiles across producer ingestion rates and partition topologies:

```
experiment,parameter,target_rate,actual_throughput,p50_latency_ms,p95_latency_ms,p99_latency_ms,lost_events,dataset
Exp1_Throughput,100_tx_sec,100,55.82,15.42,30.91,60.48,0,train_transaction.csv
Exp1_Throughput,500_tx_sec,500,56.93,15.44,30.92,60.45,0,train_transaction.csv
Exp1_Throughput,1000_tx_sec,1000,56.80,15.35,30.88,33.35,0,train_transaction.csv
Exp1_Throughput,5000_tx_sec,5000,1118.24,0.03,0.27,0.99,0,train_transaction.csv
Exp2_Partitions,1_partitions,500,59.15,15.35,29.40,59.58,0,train_transaction.csv
Exp2_Partitions,3_partitions,500,54.97,15.47,30.70,60.87,0,train_transaction.csv
Exp2_Partitions,6_partitions,500,57.43,15.43,30.66,31.92,0,train_transaction.csv
```

2. `member1_parallel_consumer_results.csv`

Contains multi-process consumer scaling metrics across partition counts:

```
partitions,consumers,total_consumed,elapsed_sec,throughput_tx_sec,scaling_efficiency,dataset
1,1,20000,1.85,10810.81,1.00,IEEE-CIS
3,3,20000,0.82,24390.24,2.26,IEEE-CIS
6,6,20000,0.68,29411.76,2.72,IEEE-CIS
12,12,20000,0.65,30769.23,2.85,IEEE-CIS
```

3. member1_clean_experiment_results.csv

Contains benchmark measurements under clean network parameters:

- Producer delivery confirmation latency (acks=0, acks=1, acks=all).
- Batch size buffer scaling (16KB, 64KB, 128KB, 256KB).
- Compression ratios and throughput for LZ4, Snappy, GZIP, None.

4. member1_experiment_results.csv

Summary comparison matrix covering end-to-end reliability, failure recovery times, and broker consumer lag.

6.3 Downstream Stream Processing Integration Outputs

For distributed stream processing engines (Apache Spark and Apache Flink), this Kafka codebase outputs streams structured for direct consumption:

Apache Spark Structured Streaming Output (Member 2 Hand-off)

- **Schema Definition:**

```
from pyspark.sql.types import DoubleType, IntegerType, LongType, StringType, StructField, StructType

transaction_schema = StructType([
    StructField("TransactionID", LongType(), True),
    StructField("isFraud", IntegerType(), True),
    StructField("TransactionDT", LongType(), True),
    StructField("TransactionAmt", DoubleType(), True),
    StructField("card1", StringType(), True),
    StructField("ProductCD", StringType(), True),
    StructField("timestamp", DoubleType(), True),
])
```

- **Output Spark DataFrame:** Parsed streaming table with micro-batch trigger windowing for model inference and feature extraction.

Apache Flink Complex Event Processing (CEP) Output (Member 3 Hand-off)

- **Stream Contract:** `DataStream<TransactionEvent>` keyed by `card1`.
- **Stateful Output:** 5-minute sliding window transaction counts, cumulative card spending velocity, and high-frequency anomaly alert triggers.

7. Codebase Script Inventory: Inputs & Outputs Matrix

Script Path	Purpose	Exact Inputs	Exact Outputs
kafka/transaction_generator/dataset_loader.py	Universal CSV parser & streamer	Raw CSV files in Datasets/	Normalized Python dict records
kafka/transaction_generator/transaction_generator.py	Synthetic event generator	Config parameters (rates, card pools)	Synthetic transaction records
kafka/producer/high_throughput_producer_benchmark.py	High-volume ingestion benchmark	Pre-cached memory records, CLI worker flags	Ingested Kafka messages, throughput/latenc stats
kafka/producer/ieee_cis_replay_producer.py	Realistic transaction replay	IEEE-CIS CSV, target replay speed	Real-time stream to topic ieee_cis_transactions
kafka/producer/batching_compression_benchmark.py	Compression & batching benchmark	Codecs (none, lz4, snappy, gzip), batch sizes	Benchmark throughput & compression ratio
kafka/producer/acks_test_producer.py	Acknowledgment durability test	acks config (0, 1, all)	Latency percentiles & delivery guarantee validation

Script Path	Purpose	Exact Inputs	Exact Outputs
kafka/consumer/fraud_detection_consumer.py	Real-time rule-based fraud detector	Kafka transactions stream	Approved/Blocked console logs & fraud aler
kafka/consumer/multiprocess_parallel_consumer_benchmark.py	Multi-core consumer scaling test	Partitioned Kafka topics (p1, p3, p6, p12)	member1_parallel_consumer_results.cs
kafka/consumer/at_least_once_consumer.py	Exactly-once / At-least-once validator	Kafka stream + simulated crashes	Duplicate counts & offset commit state
kafka/run_member1_dataset_experiments.py	Automated research suite executor	All datasets & benchmark test configurations	Generated CSV result files in workspace root
member1_results_summary.py	Benchmark summary aggregator	member1_*_results.csv files	Formatted research metric tables & terminal summaries

8. Summary of File Formats

- **Inputs:** `.csv` (Comma-separated values), CLI flags, configuration objects, synthetic generators.
- **Transit Wire Protocol:** Binary TCP over Kafka Protocol, JSON UTF-8 byte payloads, UTF-8 string routing keys.
- **Outputs:** Serialized Kafka message streams, formatted terminal decision streams, summary statistical tables, `.csv` experimental result datasets.